

DOCUMENTACION TECNICA

# Runbook: puente Hermes

ESP32-S3 Zero

[RUNBOOK-HERMES-BRIDGE.MD](#)

# Runbook: puente Hermes — ESP32-S3 Zero

## TABLA DE CONTENIDOS

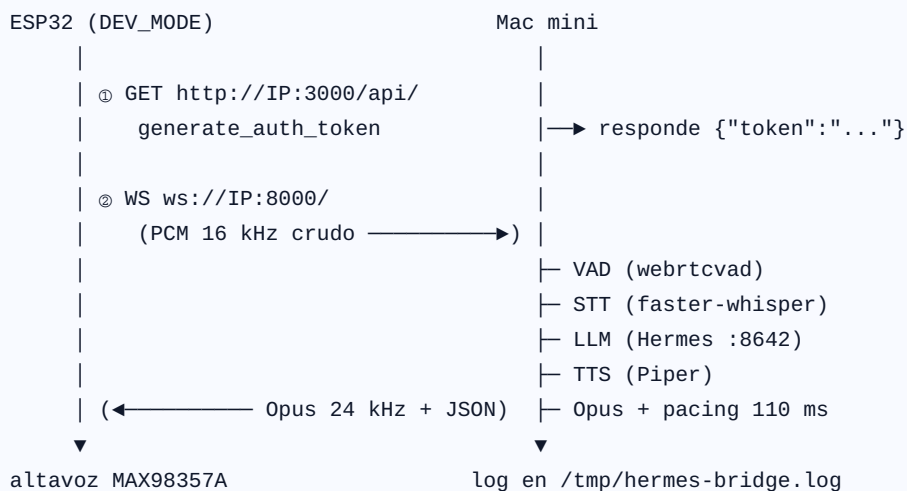
|                                                     |   |
|-----------------------------------------------------|---|
| Runbook: puente Hermes — ESP32-S3 Zero .....        | 2 |
| Descripcion general.....                            | 2 |
| Flujo de operacion final .....                      | 3 |
| Requisitos previos .....                            | 3 |
| Paso 1 — Habilitar la API de Hermes .....           | 3 |
| Paso 2 — Fijar la IP de la Mac mini .....           | 4 |
| Paso 3 — Instalar el puente .....                   | 4 |
| Paso 4 — Descargar la voz de Piper (español) .....  | 4 |
| Paso 5 — Probar el puente manualmente .....         | 5 |
| Variables de configuracion (todas opcionales) ..... | 5 |
| Paso 6 — Firmware: IP y flasheo .....               | 6 |
| Paso 7 — Verificacion end-to-end.....               | 6 |
| Paso 8 — Dejarlo como servicio (launchd) .....      | 7 |
| Rollback a la nube de Elato.....                    | 7 |
| Solucion de problemas .....                         | 8 |
| Referencia rapida.....                              | 9 |

## Descripcion general

Procedimiento completo para poner en marcha el **puente ElatoAI ↔ Hermes**: el ESP32-S3 Zero deja de hablar con la nube de Elato y pasa a usar un servidor local (Mac mini) que transcribe con faster-whisper, consulta al agente **Hermes** y sintetiza voz con Piper. Este runbook esta escrito para que **cualquier agente o persona** pueda ejecutarlo de principio a fin.

**NOTA** El codigo del puente ya existe en `server/hermes-bridge/bridge.py` y el firmware ya esta cambiado a `DEV_MODE` (commit correspondiente). Lo que queda es la instalacion en la Mac mini y ajustar la IP. El diseño y sus razones estan en `propuesta-hermes-bridge`.

## Flujo de operacion final



## Requisitos previos

- Mac mini con **Hermes instalado y funcionando** (`hermes_gateway` arranca).
- Homebrew y Python 3.10+ en la Mac mini.
- El repo ElatoAI clonado o copiado en la Mac mini (basta `server/hermes-bridge/`).
- El ESP32-S3 Zero con su montaje validado (ver `conexionado-por-componente`).
- Un equipo con PlatformIO para reflashear el firmware (paso 6).

## Paso 1 — Habilitar la API de Hermes

En la Mac mini, editar `~/hermes/.env` y añadir:

```
API_SERVER_ENABLED=true
API_SERVER_KEY=elige-una-clave-secreta
```

Reiniciar el gateway y comprobar que aparece la línea del API server:

```
hermes gateway
# esperado en el log: [API Server] API server listening on http://127.0.0.1:8642
```

Verificar con curl:

```
curl http://localhost:8642/v1/chat/completions \
-H "Authorization: Bearer elige-una-clave-secreta" \
-H "Content-Type: application/json" \
-d '{"model": "hermes-agent", "messages": [{"role": "user", "content": "hola"}]}'
```

**ADVERTENCIA** La API de Hermes da acceso a todo su toolset (incluida la terminal). **No cambiar** `API_SERVER_HOST` de su valor por defecto (`127.0.0.1`): el puente corre en la misma maquina y no necesita que Hermes escuche en la red.

## Paso 2 — Fijar la IP de la Mac mini

El firmware apunta a una IP fija. Reservar la IP de la Mac mini en el router (reserva DHCP) para que nunca cambie. Anotarla; se usa en los pasos 6 y 7.

```
# En la Mac mini, para conocer la IP actual:
ipconfig getifaddr en0 || ipconfig getifaddr en1
```

## Paso 3 — Instalar el puente

```
cd <repo>/server/hermes-bridge
brew install opus # libopus, requerido por opuslib
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
```

## Paso 4 — Descargar la voz de Piper (español)

```
mkdir -p ~/piper-voices && cd ~/piper-voices
curl -LO https://huggingface.co/rhasspy/piper-voices/resolve/main/es/es_ES/davefx/medium/es_ES-davefx-medium.
curl -LO https://huggingface.co/rhasspy/piper-voices/resolve/main/es/es_ES/davefx/medium/es_ES-davefx-medium.
```

**TIP** Cualquier voz de `rhasspy/piper-voices` sirve (hay varias en español). Si se usa otra, ajustar `PIPER_VOICE` en el paso 5. El `.onnx.json` es obligatorio: el puente lee de ahí el sample rate.

## Paso 5 — Probar el puente manualmente

```
cd <repo>/server/hermes-bridge
HERMES_API_KEY=elige-una-clave-secreta \
PIPER_VOICE=~/.pipper-voices/es_ES-davefx-medium.onnx \
.venv/bin/python3 bridge.py
```

Salida esperada (la primera vez tarda: descarga el modelo Whisper):

```
... Loading faster-whisper model 'small'...
... Whisper model loaded.
... Bridge ready: ws://0.0.0.0:8000/ token http://0.0.0.0:3000/api/generate_auth_token
... Hermes endpoint: http://127.0.0.1:8642/v1/chat/completions (model hermes-agent)
```

Verificar el endpoint del token desde otra maquina de la LAN:

```
curl "http://<IP-mac-mini>:3000/api/generate_auth_token?macAddress=TEST"
# esperado: {"token": "elato-local-token"}
```

### Variables de configuracion (todas opcionales)

| Variable             | Default                                                | Uso                                                                             |
|----------------------|--------------------------------------------------------|---------------------------------------------------------------------------------|
| HERMES_API_KEY       | (vacia)                                                | La <code>API_SERVER_KEY</code> de Hermes                                        |
| HERMES_URL           | <code>http://127.0.0.1:8642/v1/chat/completions</code> | Endpoint LLM                                                                    |
| WHISPER_MODEL        | <code>small</code>                                     | Modelo STT ( <code>base</code> = mas rapido, <code>medium</code> = mas preciso) |
| PIPER_VOICE          | <code>~/pipper-voices/es_ES-davefx-medium.onnx</code>  | Voz TTS                                                                         |
| BRIDGE_LANGUAGE      | <code>es</code>                                        | Idioma para Whisper                                                             |
| BRIDGE_SYSTEM_PROMPT | (prompt en español)                                    | Personalidad del asistente                                                      |
| VAD_SILENCE_MS       | <code>800</code>                                       | Silencio que cierra la frase                                                    |
| BRIDGE_AUTH_TOKEN    | <code>elato-local-token</code>                         | Token devuelto al ESP32 (no se valida)                                          |

## Paso 6 — Firmware: IP y flasheo

El firmware ya esta en `DEV_MODE` + `VOICE_SERVER_DENO`. Solo falta poner la IP real de la Mac mini (paso 2) en **dos líneas** de `firmware-arduino/src/Config.cpp` (bloque `#ifdef DEV_MODE`, ~líneas 43 y 54):

```
const char *ws_server = "192.168.1.50";      // <- IP de la Mac mini
...
const char *backend_server = "192.168.1.50"; // <- la misma IP
```

Compilar y flashear (el Zero necesita `--no-stub`, ya configurado; detalles en `firmware-binarios-y-flasheo`):

```
cd firmware-arduino
pio run -t upload --upload-port /dev/ttyACM0
```

**NOTA** El token de Elato guardado en NVS no estorba: en `DEV_MODE` el firmware pide un token nuevo al puente solo si NVS esta vacio, y el puente acepta cualquier token (no valida el header). No hace falta borrar NVS.

## Paso 7 — Verificacion end-to-end

1. Con el puente corriendo, encender el ESP32.
2. **Log del puente:** debe aparecer `Token request from MAC B8:F8:62:D7:59:C4` y luego `Device connected (MAC ...)`.
3. **LED:** verde/amarillo (escuchando) tras el arranque.
4. **Hablar** una frase y callar: el log muestra `User: <transcripcion>`, luego `Hermes: <respuesta>`, el LED pasa a rojo (procesa) y azul (habla), y el altavoz reproduce la respuesta.
5. El **boton KY-004** sigue funcionando igual (dormir/despertar).

Monitor serie del ESP32 (opcional): `[WSc] Connected to url: /`, luego `AUDIO.COMMITTED` / `RESPONSE.CREATED` / `RESPONSE.COMPLETE` en cada turno.

## Paso 8 — Dejarlo como servicio (launchd)

---

```
cd <repo>/server/hermes-bridge
# 1. Editar com.elato.hermes-bridge.plist: rutas reales del venv/repo,
#   HERMES_API_KEY y PIPER_VOICE (buscar "AJUSTAR").
cp com.elato.hermes-bridge.plist ~/Library/LaunchAgents/
launchctl load ~/Library/LaunchAgents/com.elato.hermes-bridge.plist
# logs:
tail -f /tmp/hermes-bridge.log
```

Con `KeepAlive` el puente se reinicia solo si se cae, y arranca al encender la Mac mini. El ESP32 reintenta la conexión WebSocket cada 1 s automáticamente, así que un reinicio del puente se recupera sin tocar la placa.

---

## Rollback a la nube de Elato

---

En `firmware-arduino/src/Config.h`, invertir los defines y reflashear:

```
// #define DEV_MODE
#define ELATO_MODE
...
// #define VOICE_SERVER_DENO
#define VOICE_SERVER_CLOUDFLARE
```

Nada más: los certificados y URLs de Elato siguen intactos en `Config.cpp`. También existe el respaldo `~/Firmware_ELATO_ok_limpio_merged.bin` (ver `firmware-binarios-y-flasheo`) que restaura la placa completa a la configuración Elato con un solo comando.

---

## Solucion de problemas

| Sintoma                                         | Causa probable                                      | Solucion                                                                      |
|-------------------------------------------------|-----------------------------------------------------|-------------------------------------------------------------------------------|
| <code>Piper voice not found</code> al arrancar  | Falta la voz o la ruta es otra                      | Paso 4; ajustar <code>PIPER_VOICE</code>                                      |
| <code>Hermes HTTP 401</code> en el log          | <code>HERMES_API_KEY</code> no coincide             | Usar la misma <code>API_SERVER_KEY</code> de <code>~/hermes/.env</code>       |
| <code>Hermes HTTP</code> con connection refused | Hermes no corre o API no habilitada                 | Paso 1; <code>hermes gateway</code> activo                                    |
| El ESP32 no pide token                          | IP mal puesta en <code>Config.cpp</code> o firewall | Verificar paso 6 y el firewall de macOS (permitir Python)                     |
| Conecta pero nunca transcribe                   | El VAD no detecta voz (ruido de fondo alto)         | Subir <code>VAD_AGGRESSIVENESS</code> a 3 o revisar el micro                  |
| Transcribe frases vacias/cortadas               | Silencio de cierre muy corto                        | Subir <code>VAD_SILENCE_MS</code> (p. ej. 1000)                               |
| Audio entrecortado en el altavoz                | Se perdio el pacing (CPU saturada)                  | Cerrar cargas pesadas; whisper <code>base</code> en vez de <code>small</code> |
| Respuestas muy lentas                           | Hermes usa herramientas (agentic)                   | Configurar <code>model_routes</code> en Hermes para respuestas directas       |
| <code>ImportError: opuslib</code>               | Falta libopus                                       | <code>brew install opus</code>                                                |
| El puente muere al desconectar el ESP32         | — (no debe pasar; reconexion soportada)             | Revisar <code>/tmp/hermes-bridge.err</code> y reportar                        |



## Referencia rápida

### MAC MINI:

```
~/hermes/.env          -> API_SERVER_ENABLED=true, API_SERVER_KEY=...
server/hermes-bridge/  -> bridge.py + requirements.txt + plist
~/piper-voices/        -> voz TTS (.onnx + .onnx.json)
puertos: 3000 (token HTTP), 8000 (WS audio), 8642 (Hermes, solo local)
```

### FIRMWARE (firmware-arduino/):

```
src/Config.h          -> #define DEV_MODE + #define VOICE_SERVER_DENO
src/Config.cpp         -> ws_server y backend_server = IP de la Mac mini
flasheo                -> pio run -t upload --upload-port /dev/ttyACM0
```

### SERVICIO:

```
launchctl load ~/Library/LaunchAgents/com.elato.hermes-bridge.plist
tail -f /tmp/hermes-bridge.log
```

ROLLBACK: Config.h -> ELATO\_MODE + VOICE\_SERVER\_CLOUDFLARE y reflashear.